

Layout Patterns with BPMN Semantics

Philip Effinger

Wilhelm-Schickard-Institut für Informatik,
Eberhards Karls Universität Tübingen, Germany
philip.effinger@uni-tuebingen.de

Abstract. BPMN is a notation language that provides visual elements for modeling business processes. The resulting BPMN diagrams that represent BPMN models follow rules concerning their layout for creating a common understanding among all BPMN designers. A subset of the rules is determined by the BPMN standard. Other rules evolved when the BPMN community gained experience in the usage of its notation language. From a layout algorithmic perspective, the rules are formalized as so-called aesthetics. Until today, aesthetics for BPMN are mostly limited to the underlying graph structure of a BPMN process model diagram.

In this work, we present new layout patterns that can be applied in layout applications of BPMN and its modeling tools. The new layout patterns support BPMN semantics and address layout issues that are not covered by aesthetics so far. The patterns can be combined to achieve positive effects on multiple a) layout issues and b) BPMN semantics at a time. We also give detailed algorithmic descriptions for our patterns.

1 Motivation

For many users of BPMN (> 50%, according to a survey among experts¹), the main purpose of BPMN is *Process Description*. The description of a process employs the graphical notation of BPMN. As BPMN specifies graphical representations for its elements, these given representations quickly allow simple diagramming in modeling tools when developing a model.

Our focus in this work is on the support of a user when graphically developing a process model. We aim to provide features in tools that can create a layout for a given process diagram. The layout of a diagram should be created automatically (by a single mouse-click) and it is expected to appeal to the user. The layout methods are based on graph-drawing algorithms.

In previous work, we presented a survey on user requirements towards BPMN diagrams and specific layout properties of BPMN models [1]. From the survey, we can derive important expectations and criteria for layout, so-called aesthetics. Also, there exist approaches for layout support in BPMN ([2,3]). The existing layout approaches are based on the structure of a BPMN diagram. The structure is considered to be a graph with nodes (BPMN elements) and edges connecting

¹ <http://bpex.blogspot.com/2011/05/bpmn-usage-survey-results.html>

the nodes (in BPMN: sequence flow, conditional flow, default flow). However, the graph structure does not consider the types of the nodes (BPMN semantics).

In this work, we present patterns that are designed to overcome the gap between graph structure and semantics of BPMN diagrams when creating a layout. The patterns aim to a) reduce cluttering in diagrams (primarily around gateways), b) highlight the logical structure of a BPMN diagram (induced by gateways) and c) accentuate the process flow.

2 Layout Patterns

The new layout patterns aim to reduce the lack of BPMN semantics in existing BPMN layout approaches [2,3]. Inspecting the list of standard layout aesthetics of [1], it is easy to observe that BPMN semantics in terms of element types are not taken into account in any of the aesthetics. The lack of support for BPMN element types is a gap between the graph structure and BPMN semantics. By introducing new layout patterns in the next section, we try to take a first step to overcome this syntax-semantics gap in BPMN layout. The patterns should be considered as a starting point for BPMN semantics in layout approaches and cannot be considered to be complete. For instance, one pattern might, on the one hand, enlarge the area size of the diagram and insert a new crossing between two flows, but on the other hand, three bends are removed and the length of a flow is reduced. This shows the tradeoff between an aesthetically optimal solution and consideration of semantics.

2.1 Geometric Pattern (GeoP)

Description: The Geometric Pattern (GeoP) aims at reducing visual cluttering. Cluttering describes the occurrence of many elements in a small amount of the diagram area (high element-density). For instance, around gateways with multiple connected elements, the cluttering is higher than the cluttering around a start event with a single outgoing sequence flow. Therefore, reducing the visual cluttering demands a reduction of density.

As a pre-processing step, we will show how the density of a BPMN diagram is determined. Density values can be visualized as depicted in Figure 1a). The calculation defines densities diametrically opposed to the flow in the model for each swimlane. The density values are derived by performing a sweep-line algorithm [4] call for each swimlane individually. In a single sweep, an event handler changes the density counter depending on incoming events. In our case, events are left/right geometric sides of boxes representing BPMN elements.

After the detection of dense areas, we insert temporary edges (marking edges) in the center of each dense area. The edges mark these areas for the algorithm. The pre-processing step ends with the edges' insertion orthogonally to the swimlane orientation, see Figure 1b). The algorithm of the pattern then aims at resolving the introduced overlaps by moving (sifting) nodes affected by marking edges.

Algorithm: Our algorithm for GeoP is related to the *Sifting Algorithm (SA)* [5,6]. In general, SA tries to move one element at a time along an ordering of

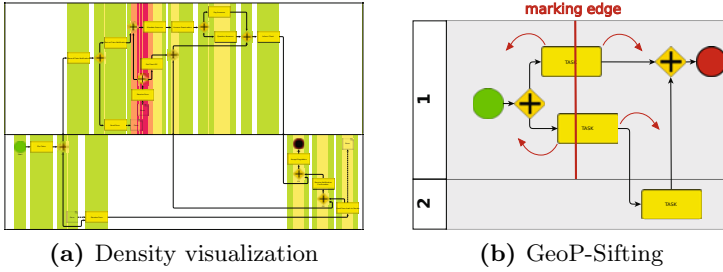


Fig. 1. (a) Visualization of densities (red = high, green = low). (b) Schematic view of the algorithm for the GeoP. Red arrows depict the direction of possible sifting moves.

other elements until a goal function reaches a (local) minimum. In our case, we move all elements sequentially that are overlapped by a temporary marking edge. An element can be moved in parallel to its swimlane. Therefore, other than in the general SA, we have two orderings that an element can be moved along, see Figure 1b). Thus, we have more freedom and move an element in either direction until the overlap with the marking edge is prevented. This spreads the original density center and the cluttering is reduced.

To reduce cluttering in several dense areas, the algorithm allows to insert multiple temporary edges that are processed sequentially.

2.2 Gateway Pattern (GaP)

Description: The Gateway Pattern (GaP) aims at highlighting the logical structure of a BPMN process model. The logic structure is defined by the combination of gateways. Gateways determine the process flow based on logical expressions that are evaluated when a gateway is passed. Evaluations of gateways may cause split/joins of process flow(s). Since BPMN is not a block-structured notation language, but a graph-based notation language [7], the underlying logic structure is not trivial. In [8], the challenges of 'unraveling' (transforming) a non well-structured process model to a well-structured model are described. In general, if a process model is not well-structured, determination of corresponding pairs of opening split and closing joins is not unique. We call such a pair 'GaP-pair'.

If we can find such a GaP-pair that encloses a block structure, GaP is implying that no element of this block is placed 'before' the opening split gateway (with respect to the process model flow orientation) or 'after' the closing join gateway. Thus, the goal of GaP is to highlight the semantic block structure by a visual block structure that is surround by the GaP-pair.

In most process models, we cannot find well-structured parts when analysing the gateways. Therefore, we apply a method that allows the construction of blocks that do not require well-structured processes. The details of this algorithm are described in the following.

Algorithm: For our approach, we use the concept of a path in a graph: A path is a sequence of consecutive edges in a graph and the length of the path is the number of edges traversed. Since we also want to handle non-well-structured

models, we define a GaP-pair as follows: A split-gateway $G1$ and a join-gateway $G2$ form a GaP-pair, if, for all possible paths starting at $G1$, the size of the subset of paths arriving at $G2$ is maximal among all gateways. In other words, we count the number of paths arriving at any gateway, where $G1$ is the root node in the path; then, for $G2$, the counter reaches its maximum and $G1$ and $G2$ form a GaP-pair.

Counting paths is executed using a variant of depth-first-search (DFS) [9]: we start a DFS-run from every split-gateway (or other gateways that perform process flow split, e.g. complex gateways). Since DFS is able to handle cycles (by storing visited nodes) and we employ our path counting method, we are able to find GaP-pairs in non-well-structured models.

If, for a split-gateway $G1$, there are two join-gateways $G2a$ and $G2b$ with the same path counter, the minimum distance between $G1$ and either $G2a$ or $G2b$ is taken as criteria for determining the GaP-pair.

After finding GaP-Pairs, we insert surrounding temporary edges (skeleton edges), analogously to the GeoP, orthogonally to the flow orientation. Skeleton edges are inserted into the swimlane(s) of the gateways forming the GaP-pairs, see Figure 2. For every pair, one skeleton edge is introduced before the split-gateway and a second skeleton edge after the join-gateway. The skeleton edges delimit the surrounding box representing the block that is formed by the GaP-pair. When performing a layout (for instance with the approach presented in [3]), the skeleton edges prevent nodes to be moved to the outside of the block.

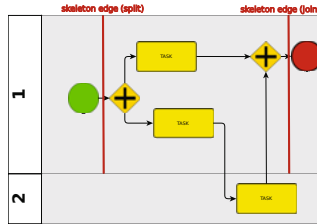


Fig. 2. Insertion points of skeleton edges in GaP

2.3 Start-End-Pattern (SEP)

Description: The Start-End-Pattern (SEP) formalizes the compliance of placing start- and end-events considering aesthetics FLOW [1]. These elements should be placed such that they follow the orientation of the process flow (or 'reading' direction of the user). When SEP is activated, it ensures that a start-event is moved to the 'beginning' (left side) of its assigned swimlane, and an end-events is set to the 'end' (right side) its swimlane. Performing SEP might affect lengths of edges: edges connected to moved events might increase severely in length. In order to possibly reduce the increase of edge length or prevent bends, we support two variants of SEP:

1. **Dynamic:** Events affected by SEP are set to the left/right border of the swimlanes but may move in parallel to the swimlane orientation in order to reduce unnecessary long edges or prevent bends, see Figure 3a).

2. **Locking:** All (start/end) events of a swimlane are aligned in the beginning/end of the swimlane, see Figure 3b). The events are locked in a box surrounded by skeleton edges that guarantee that they do not move to the outer side of the box when performing a new layout.

The second variant is more appropriate for process models that have highly parallel process flow and, thus, several starting/terminating events.

Algorithm: As mentioned in the two variants, we employ for SEP the idea of skeleton edges that was also used in GaP. The skeleton edges ensure the structure of the locking box and keep the nodes aligned in the beginning/end of the swimlanes. The alignment of the event node of one swimlane is done such that overlapping of nodes is prevented and the size of an element is kept.

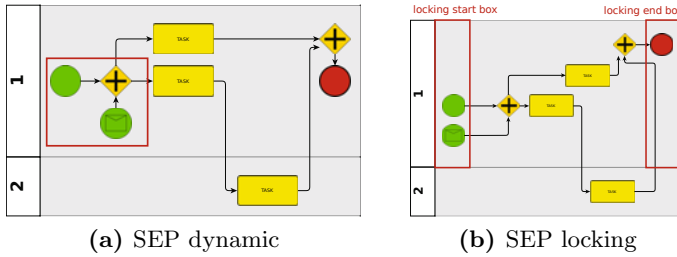


Fig. 3. (a) Start- and End-events are allowed to optimize their position, e.g. see the node MESSAGE_START_EVENT that moved in order to prevent a bend in a flow. (b) Start- and End-events are kept static in boxes built by skeleton edges.

3 Related Work

Patterns in (business) processes are a common method to express abstract similarities in process models, e.g., *action patterns* are used to formalize semantic analysis of BPMN process models in [10] or for organization in large business process model repositories [11]. The term 'layout patterns' is used in [12] for expressing layout constraints that represent conditions for layout algorithms.

For model decomposition that was performed in Section 2 when analysing the graph for GaP-pairs, a preferred decomposition for process models is often using SPQR-trees for a subsequent analysis of the structure or application of rules for abstraction [13].

4 Conclusion

In this work, we introduce new layout patterns for BPMN diagrams. The layout patterns allow to include BPMN semantics in layout algorithms for BPMN process models. The layout patterns take into account common rules of BPMN diagrams, so-called aesthetics. The patterns address the following issues in BPMN diagrams: a) Cluttering of nodes in diagrams, b) Perceiving the logical structure of a BPMN diagram (induced by gateways) and c) Accentuate the process flow at start events and process' termination in end events.

The presented set of layout patterns is not to be considered complete but constitutes a starting point for rising interest in BPMN semantics in layout approaches in future (commercial) applications. At the moment, we are conducting an empirical evaluation with an available set of more than 750 process models [14]. Furthermore, it will be an interesting task for the future to conduct a user study in order to receive feedback on the human aesthetics' perspective on the drawbacks/benefits of the presented patterns. Also, statements from users might lead to additional patterns that incorporate more BPMN semantics in layouts.

References

1. Effinger, P., Jogsch, N., Seiz, S.: On a Study of Layout Aesthetics for Business Process Models Using BPMN. In: Mendling, J., Weidlich, M., Weske, M. (eds.) BPMN 2010. LNBIP, vol. 67, pp. 31–45. Springer, Heidelberg (2010)
2. Kitzmann, I., König, C., Lübke, D., Singer, L.: A Simple Algorithm for Automatic Layout of BPMN Processes. In: CEC, pp. 391–398 (2009)
3. Effinger, P., Siebenhaller, M., Kaufmann, M.: An Interactive Layout Tool for BPMN. In: IEEE Internat. Conference on E-Commerce Technology, vol. 1, pp. 399–406 (2009)
4. Bentley, J.L., Ottmann, T.: Algorithms for reporting and counting geometric intersections. *IEEE Trans. Computers* 28(9), 643–647 (1979)
5. Matuszewski, C., Schönfeld, R., Molitor, P.: Using Sifting for k -Layer Straightline Crossing Minimization. In: Kratochvíl, J. (ed.) GD 1999. LNCS, vol. 1731, pp. 217–224. Springer, Heidelberg (1999)
6. Rudell, R.: Dynamic variable ordering for ordered binary decision diagrams. In: Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD 1993), pp. 42–47 (1993)
7. Kopp, O., Martin, D., Wutke, D., Leymann, F.: The difference between graph-based and block-structured business process modelling languages. *Enterprise Modelling and Information Systems Architectures* 4(1), 3–13 (2009)
8. Dumas, M., García-Bañuelos, L., Polyvyanyy, A.: Unraveling Unstructured Process Models. In: Mendling, J., Weidlich, M., Weske, M. (eds.) BPMN 2010. LNBIP, vol. 67, pp. 1–7. Springer, Heidelberg (2010)
9. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms, 2nd edn. The MIT Press (September 2001)
10. Smirnov, S., Weidlich, M., Mendling, J., Weske, M.: Action Patterns in Business Process Models. In: Baresi, L., Chi, C.-H., Suzuki, J. (eds.) ICSOC-ServiceWave 2009. LNCS, vol. 5900, pp. 115–129. Springer, Heidelberg (2009)
11. Smirnov, S., Weidlich, M., Mendling, J., Weske, M.: Object-sensitive action patterns in process model repositories. In: BPM Workshops, pp. 251–263 (2010)
12. Maier, S., Minas, M.: Interactive diagram layout. In: Proceedings of the 28th Internat. Conference on Human factors in Computing systems, CHI EA 2010, pp. 4111–4116. ACM, NY (2010)
13. Polyvyanyy, A., Smirnov, S., Weske, M.: The Triconnected Abstraction of Process Models. In: Dayal, U., Eder, J., Koehler, J., Reijers, H.A. (eds.) BPM 2009. LNCS, vol. 5701, pp. 229–244. Springer, Heidelberg (2009)
14. Fahland, D., Favre, C., Jobstmann, B., Koehler, J., Lohmann, N., Völzer, H., Wolf, K.: Instantaneous Soundness Checking of Industrial Business Process Models. In: Dayal, U., Eder, J., Koehler, J., Reijers, H.A. (eds.) BPM 2009. LNCS, vol. 5701, pp. 278–293. Springer, Heidelberg (2009)